

Sprint 1: Tính năng Profile & Cấu hình nền tảng

Trong Sprint này, Frontend (FE) và Backend (BE) sẽ làm việc song song trên cùng một tính năng. Đến cuối Sprint, nhóm sẽ có một trang Cài đặt hoàn chỉnh: thay đổi được Tên, chuyển đổi Theme Sáng/Tối và lưu trữ thành công xuống file cấu hình.

Thẻ 1: [2h] Khởi tạo dự án & Thống nhất dữ liệu Profile

 **Người phụ trách:** Cả team (PM chủ trì)

 **Mô tả:**

Tạo không gian làm việc chung, dựng base code và chốt chính xác các trường dữ liệu sẽ giao tiếp giữa FE và BE cho tính năng Profile.

Checklist:

- ☐ **Github:** Tạo Repository chung, phân quyền cho thành viên.
- ☐ **Base BE:** BE khởi tạo dự án Node.js/Express (npm init), cài cors, đưa code lên nhánh main.
- ☐ **Base FE:** FE dùng Vite/Create React App tạo dự án React, đẩy lên thư mục frontend trên repo.
- ☐ **Chốt JSON:** Thống nhất format file profile.json gồm các trường: displayName (chuỗi), theme (chuỗi: 'light'|'dark'), password (chuỗi - dùng cho Sprint sau).

Thẻ 2: [4h] (BE) API Profile - Đọc và Cập nhật cấu hình

 **Người phụ trách:** Backend Dev

 **Mô tả:**

Viết API bằng Node.js và module fs để đọc/ghi thông tin cấu hình vào file data/profile.json.

Checklist:

- ☐ Tạo thư mục data/ và tạo sẵn file profile.json với dữ liệu mặc định (VD: tên trống, theme sáng).
- ☐ Viết API **GET** /api/profile: Dùng fs.readFileSync đọc file và trả về JSON cấu hình (Chưa cần trả về password).
- ☐ Viết API **PUT (hoặc POST)** /api/profile: Nhận cục JSON từ FE (Tên, Theme, Password) truyền lên.

- [] Xử lý logic ghi đè (update) bằng `fs.writeFileSync`.
- [] Test bằng Postman: Bắn thử API GET và PUT. Chụp ảnh màn hình Postman đính kèm vào thẻ này kèm JSON mẫu để FE bắt đầu gọi API.

Thẻ 3: [4h] (FE) Khởi tạo Layout, Router & Global State (Theme)

 **Người phụ trách:** Frontend Dev

 **Mô tả:**

Thiết lập bộ khung giao diện của ứng dụng. FE không cần đợi API của BE làm xong, cứ dùng Mock Data (dữ liệu giả) để làm bước này trước.

Checklist:

- [] Cài đặt react-router-dom, cấu hình 3 route trống: / (Home), /settings (Cài đặt), /private (Vùng kín).
- [] Vẽ Layout chung: Sidebar (chứa các link điều hướng) và Content (vùng hiển thị trang).
- [] Tạo AppContext (Context API) để lưu trạng thái toàn cục: displayName và theme ('light' / 'dark').
- [] Viết logic CSS/Tailwind cơ bản để khi state theme thay đổi, màu nền của Layout sẽ đổi theo.

Thẻ 4: [5h] (FE) Màn hình Cài đặt & Ghép API Profile

 **Người phụ trách:** Frontend Dev

 **Mô tả:**

Tạo Form cấu hình và tiến hành ghép nối với API từ Backend để dữ liệu thực sự được lưu xuống máy chủ.

Checklist:

- [] Tạo Form ở trang /settings gồm: Input Tên hiển thị, Radio/Select chọn Theme Sáng/Tối, Input thiết lập mật khẩu vùng kín.
- [] **Lúc load trang:** Trong useEffect, gọi API **GET** /api/profile -> Đẩy dữ liệu lấy được vào Form và vào AppContext.
- [] **Lúc bấm Lưu:** Bắt sự kiện submit Form -> Gọi API **PUT** /api/profile để gửi dữ liệu mới lên BE.

- [] Nếu API báo thành công: Cập nhật lại AppContext ngay lập tức để giao diện đổi màu (không cần F5) + Hiện thông báo "Lưu thành công".

Thẻ 5: [2h] (PM) Nghiệm thu tính năng (Testing Sprint 1)

 **Người phụ trách:** PM

 **Mô tả:**

Khi BE và FE đã hoàn tất 4 thẻ trên, PM sẽ đóng vai trò người dùng cuối để kiểm tra luồng chạy từ đầu đến cuối xem có lỗi không.

Checklist:

- [] Lấy code mới nhất từ Github về, chạy npm start cho cả BE và FE.
- [] Vào web, thao tác đổi Tên, đổi Theme sang Tối, đặt Mật khẩu và bấm Lưu.
- [] Nhìn UI: Giao diện có đổi màu ngay lập tức không? Tên ở Sidebar có đổi theo không?
- [] Nhìn Data: Tắt trình duyệt, mở lại (F5) xem giao diện có giữ nguyên màu tối không? Mở file data/profile.json ở Backend xem dữ liệu đã được ghi đúng chưa?
- [] Nếu có lỗi: Comment vào thẻ này, tag @FE hoặc @BE để sửa. Nếu tốt: Kéo toàn bộ thẻ Sprint 1 sang cột Done!

Thẻ 6: [2h] (PM) Lập Kế hoạch Test & Tạo Template Testcase

- **Vị trí:** Đặt vào cột công việc của Sprint 1 (có thể làm song song lúc Dev đang khởi tạo code).
- **Người phụ trách:** PM
-  **Checklist:**
 - [] Tạo một file Google Sheets dùng chung cho cả nhóm.
 - [] Tạo các cột chuẩn cho Testcase: Mã TC, Tên chức năng, Các bước (Steps), Kết quả mong đợi (Expected), Trạng thái (Pass/Fail/Untested), Ghi chú.
 - [] Viết sẵn 2-3 Testcase mẫu cho phần "Cài đặt Profile" (Đổi tên, Đổi Theme).
 - [] Gắn link file Google Sheets này vào phần mô tả của bảng Trello để ai cũng bấm vào xem được.

Hướng dẫn thực hiện Sprint 1: Khởi tạo và Tính năng Profile.

Để dễ quản lý, chúng ta sẽ cấu trúc dự án chung trong một thư mục gốc (Repository), bên trong chứa 2 thư mục con là backend và frontend.

PHẦN 1: KHỞI TẠO REPOSITORY CHUNG TRÊN GITHUB (Cả team)

1. **Project Manager (PM)** lên Github tạo một Repository mới (ví dụ: do-an-ghi-chu).
2. PM thêm các thành viên vào repo (Settings -> Collaborators).
3. Tất cả thành viên mở Terminal/CMD, clone code về máy tính:

```
git clone https://github.com/ten-user/do-an-ghi-chu.git
cd do-an-ghi-chu
```

4. **Tạo file .gitignore ở thư mục gốc** để tránh đẩy file rác lên Git. Tạo file .gitignore và dán nội dung sau vào:

```
node_modules/
.env
dist/
```

PHẦN 2: THỰC HIỆN BACKEND (Dành cho Backend Dev)

Bước 1: Khởi tạo Backend

Trong thư mục gốc do-an-ghi-chu, BE Dev mở Terminal và gõ:

```
mkdir backend
cd backend
npm init -y
npm install express cors
```

Bước 2: Tạo dữ liệu mẫu

Tạo thư mục data và file profile.json (backend/data/profile.json) với nội dung:

```
{
  "displayName": "Sinh viên",
  "theme": "light",
  "password": ""
}
```

Bước 3: Viết API trong server.js

Tạo file server.js trong thư mục backend và dán đoạn mã sau:

```
const express = require('express');
const cors = require('cors');
```

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

```
const fs = require('fs');
const path = require('path');
const app = express();
app.use(cors()); // Cho phép FE gọi API
app.use(express.json()); // Đọc dữ liệu JSON từ FE gửi lên
const profilePath = path.join(__dirname, 'data', 'profile.json');
// API 1: Đọc thông tin Profile
app.get('/api/profile', (req, res) => {
  try {
    const rawData = fs.readFileSync(profilePath, 'utf8');
    const profile = JSON.parse(rawData);
    res.json(profile);
  } catch (error) {
    res.status(500).json({ message: "Lỗi đọc file" });
  }
});
// API 2: Cập nhật Profile
app.put('/api/profile', (req, res) => {
  try {
    const newProfile = req.body;
    // Ghi đè dữ liệu mới vào file
    fs.writeFileSync(profilePath, JSON.stringify(newProfile, null, 2), 'utf8');
    res.json({ success: true, message: "Đã cập nhật Profile" });
  } catch (error) {
    res.status(500).json({ message: "Lỗi ghi file" });
  }
});
const PORT = 5000;
app.listen(PORT, () => console.log(`Backend chạy tại http://localhost:${PORT}`));
```

Gõ lệnh `node server.js` để chạy thử BE.

PHẦN 3: THỰC HIỆN FRONTEND (Dành cho Frontend Dev)

Bước 1: Khởi tạo Frontend

Mở 1 Terminal khác ở thư mục gốc `do-an-ghi-chu`, FE Dev gõ lệnh:

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install react-router-dom
```

Bước 2: Viết giao diện Settings (Cài đặt) và Gọi API

Do không gian có hạn, dưới đây là logic code lõi để gọi API cho trang Settings (frontend/src/Settings.jsx). Cấu trúc Router bạn có thể tự thiết lập.

```
import React, { useState, useEffect } from 'react';
function Settings() {
  const [profile, setProfile] = useState({ displayName: '', theme: 'light', password: '' });
  // Lấy dữ liệu khi vừa load trang
  useEffect(() => {
    fetch('http://localhost:5000/api/profile')
      .then(res => res.json())
      .then(data => {
        setProfile(data);
        // Đổi màu nền tạm thời dựa theo theme
        document.body.style.backgroundColor = data.theme === 'dark' ? '#333' : '#fff';
        document.body.style.color = data.theme === 'dark' ? '#fff' : '#000';
      });
  }, []);
  // Hàm xử lý khi gõ vào Input
  const handleChange = (e) => {
    setProfile({ ...profile, [e.target.name]: e.target.value });
  };
  // Hàm xử lý Lưu thay đổi
  const handleSave = () => {
    fetch('http://localhost:5000/api/profile', {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(profile)
    })
      .then(res => res.json())
      .then(data => {
        alert("Lưu thành công!");
        // Áp dụng màu nền ngay lập tức
        document.body.style.backgroundColor = profile.theme === 'dark' ? '#333' : '#fff';
        document.body.style.color = profile.theme === 'dark' ? '#fff' : '#000';
      });
  };
  return (
    <div style={{ padding: '20px' }}>
      <h2>Cài đặt hệ thống</h2>
      <div>
```

HỆ THỐNG QUẢN LÝ GHI CHÚ VÀ THÔNG TIN RIÊNG TƯ

```
        <label>Tên hiển thị: </label>
        <input name="displayName" value={profile.displayName} onChange={handleChange}
/>
    </div>
    <div style={{ marginTop: '10px' }}>
        <label>Giao diện: </label>
        <select name="theme" value={profile.theme} onChange={handleChange}>
            <option value="light">Sáng</option>
            <option value="dark">Tối</option>
        </select>
    </div>
    <div style={{ marginTop: '10px' }}>
        <label>Mật khẩu vùng kín: </label>
        <input type="password" name="password" value={profile.password}
onChange={handleChange} />
    </div>
    <button onClick={handleSave} style={{ marginTop: '20px' }}>Lưu thay
đổi</button>
</div>
);
}
export default Settings;
```

Gõ lệnh `npm run dev` để chạy thử FE.

PHẦN 4: LƯU TRỮ VÀ ĐẨY CODE LÊN GITHUB (Quy trình Commit)

Khi FE và BE đã chạy thử thành công trên máy tính, cả team tiến hành đẩy code lên Github.

Mở Terminal tại **thư mục gốc** (do-an-ghi-chu) và gõ lần lượt các lệnh sau:

1. Kiểm tra trạng thái file bị thay đổi:

```
git status
```

(Bạn sẽ thấy thư mục frontend/ và backend/ màu đỏ, không được phép có node_modules ở đây. Nếu thấy có node_modules, hãy kiểm tra lại file .gitignore).

2. Đưa tất cả thay đổi vào vùng chờ (Staging):

```
git add .
```

3. Ghi lại các thay đổi (Commit):

Nên ghi rõ nội dung để team dễ theo dõi.

```
git commit -m "feat: Hoàn thành Sprint 1 - Tính năng Profile và Setup dự án"
```

4. Đẩy code lên Github (Push):

```
git push origin main
```

PM hoặc các thành viên khác chỉ cần lên Github, pull (kéo) code mới nhất về bằng lệnh `git pull origin main` là có thể thấy ngay toàn bộ thành quả của Sprint 1 để chuẩn bị làm tiếp Sprint 2